

TYP03 × BUSINESS AVIATION

Fly the Upgrade

How pilots would fly a TYP03 upgrade

Stefan Bürk TYP03 Core Team – Framework Maintainer · stefan@buerk.tech

Today's routing

BRIEFING

Why aviation has checklists — and why you need them

CROSS-CHECK

The Core Changelog: your NOTAM package

PHASE 1

Dispatch & Pre-Flight — everything before you touch composer

PHASE 2

Take-Off & Cruise — the upgrade window itself

PHASE 3

Approach, Landing & Debrief — the part everyone skips

DEBRIEF

Takeaways you can use on Monday

Stefan Bürk

- ◆ PHP developer since 2003
- ◆ TYPO3 developer since 2012
- ◆ TYPO3 Core Team – Framework Maintainer & Merger
- ◆ Maintainer of typo3/testing-framework
- ◆ Fluid maintainer
- ◆ Aviation addict

What this is (not) about

This is not

- ◆ A copy-paste, step-by-step upgrade script
- ◆ A claim that one specific tool solves it
- ◆ A promise that nothing will go wrong

This is

- ◆ A repeatable process built from phases
- ◆ Checklists you can actually take to work
- ◆ Where each tool fits – and where it stops
- ◆ How to prepare for what you cannot foresee

Checklists were not invented by careful people.

They were invented after a crash.

On 30 October 1935 the Boeing Model 299 — the B-17 prototype — crashed on take-off at Wright Field because the gust locks were still engaged. The crew were among the best pilots in the country. The aircraft was simply more complex than memory could hold. The answer was not better pilots. The answer was the checklist.

A TYPO3 upgrade is more complex than your memory.

That is not a criticism. That is the job description.

Nobody holds 526 changelog entries, 40 extensions, a PHP jump, a schema migration and a rollback plan in their head at 23:00 on a Thursday.

"The TYP03 upgrade is on your task list."



Dread

Unknown scope. Unknown errors. Unknown hours. A weekend you will not get back.

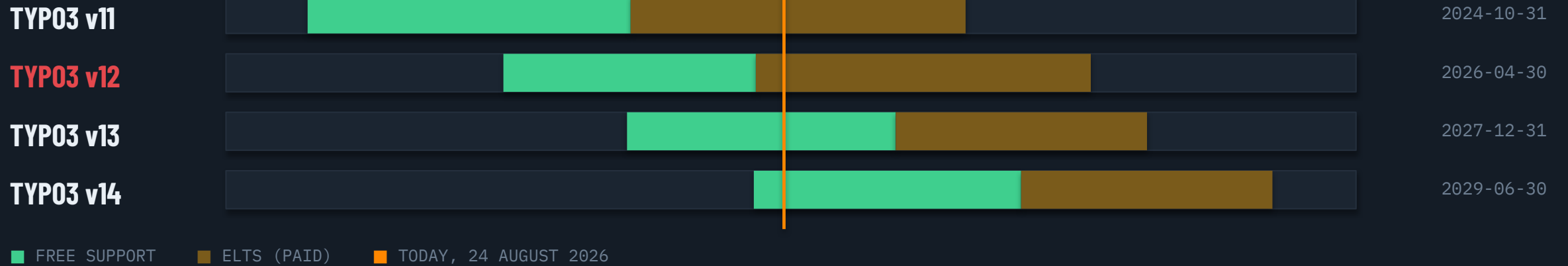


Anticipation

Known scope. Expected errors. Planned hours. A procedure you have flown before.

The difference is not talent. It is preparation.

Where the fleet stands today



Free support for v12 ended on 30 April 2026. If you are still on v12 today, you are flying on ELTS reserves — legal, paid for, and not a long-term plan.

Your dispatcher already noticed

composer refuses to release the aircraft

```
$ ddev composer require webvision/flight-ops -W
```

Problem 26

- Root composer.json requires webvision/flight-ops @dev
-> satisfiable by webvision/flight-ops[dev-main].
- webvision/flight-ops dev-main requires typo3/cms-core ^12.4
-> found typo3/cms-core[v12.4.0, ..., v12.4.45] but these were not loaded, because they are affected by security advisories ("PKSA-2yr3-by9d-r1gh", "PKSA-q3vc-jr63-rrrj", "PKSA-gj1k-954p-nhmk", "PKSA-gr4f-6g49-cg8v", "PKSA-32mm-z25f-2ysj", ... 25 more).

Installation failed, reverting ./composer.json and ./composer.lock

- ◆ This is real output from the demo instance for this talk, on a fully patched v12.4.45.
- ◆ Once free support ends, the last public release stays frozen while advisories keep landing.
- ◆ You can silence it. You cannot fix it without ELTS — or an upgrade.

Three phases, one flight

1

Pre-Flight

DISPATCH, BRIEFING, WALKAROUND

Everything before you touch composer

2

Flight

TAKE-OFF, CRUISE, THE STERILE COCKPIT

The upgrade window itself

3

Post-Flight

LANDING, TECH LOG, DEBRIEF

Verification and the next flight's plan

THE CORE CHANGELOG RUNS UNDERNEATH ALL THREE

The mapping

AVIATION

NOTAM briefing

Airworthiness Directive with a compliance date

Runway closed / navaid out of service

Pre-flight walkaround

Performance calculation

MEL — Minimum Equipment List

Departure briefing

V1 — decision speed

Sterile cockpit below 10,000 ft

QRH — Quick Reference Handbook

Stabilised approach criteria → go-around

Autopilot quietly trimming out an imbalance

Tech log write-up

TYP03 UPGRADE

Core Changelog: Breaking, Deprecation, Feature, Important

Deprecation — flies today, grounded later

Breaking change — replan or do not depart

Is the current version clean? Zero known deprecations

composer why-not, platform and PHP constraints

Findings you accept deliberately, in writing, with a date

The written plan, shared with the whole team

Schema migration and wizards: the point of no return

Code and content freeze during the window

Prepared responses for the errors you expect

Written rollback criteria, executed without ego

TCA auto-migration — it keeps flying, and it hides the problem

Deprecation log, debrief, and the next upgrade's input



YOUR NOTAM PACKAGE

The Changelog

Read before every flight. Not once, not sometimes. Every flight.

What a NOTAM actually is

- ◆ A NOTAM – Notice to Air Missions – tells a crew what changed since last time.
- ◆ Runway 07 closed. Navaid out of service. New obstacle 1.2 NM short of threshold.
- ◆ It is not interesting reading. It is mandatory reading.
- ◆ No crew departs on the assumption that the airport is the same as last month.

Yet we routinely upgrade TYP03 on exactly that assumption.

Four types, four different demands

Breaking

RUNWAY CLOSED

Something was removed or changed incompatibly.

Fix before you can run on the new version. No grace period.

Deprecation

AIRWORTHINESS DIRECTIVE WITH A COMPLIANCE DATE

Still works — and is already scheduled for removal.

Fix now, on your schedule, or it becomes a Breaking change later.

Feature

NEW EQUIPMENT FITTED

New API or capability you may adopt.

Optional — but often the migration target of a deprecation.

Important

OPERATIONAL BULLETIN

Behaviour changed in a way you should know about.

Read it. It is where the surprises hide.

The size of the briefing package

Changelog entries shipped in typo3/cms-core

TYP03 v11

351 entries



TYP03 v12

526 entries



■ BREAKING ■ DEPRECATION ■ FEATURE ■ IMPORTANT

Counted in the Documentation/Changelog folder of the shipped core. You do not read all of them — you triage them. That is a skill, and it is the job.

Anatomy of an entry

Deprecation: #96136 – Deprecate inline JavaScript in backend update signals

Description

What changed and why.

Impact

What breaks, and what it will do instead.

Affected Installations

Whether this is even your problem.

Migration

The concrete replacement. This is the response half of the checklist.

“... is deprecated and will be ignored in TYPO3 v13.0.”

```
typo3/cms-core, Documentation/Changelog/12.0/Deprecation-96136-  
DeprecateInlineJavaScriptInBackendUpdateSignals.rst
```

Every entry carries its own migration instructions. The work is reading them, not inventing them.

Today's Deprecation is tomorrow's Breaking change.

Deprecated in 12.0 → removed in 13.0. The changelog told you two years in advance.

This is why deprecation is not a phase-1 task. It is checked in all three phases, for three different reasons.

Deprecation is checked in every phase

1 Pre-Flight

Which deprecations of my **CURRENT** version am I still using?

They are the removals of the version you are flying to. Clear them while the old version still runs and still forgives you.

```
Extension Scanner · rector --dry-run
```

2 Flight

Which of them actually bite now that they are gone?

What was a warning yesterday is a fatal error today. The scanner now ships the target version's rules.

```
Extension Scanner (v13 rules) · rector ·  
fractor
```

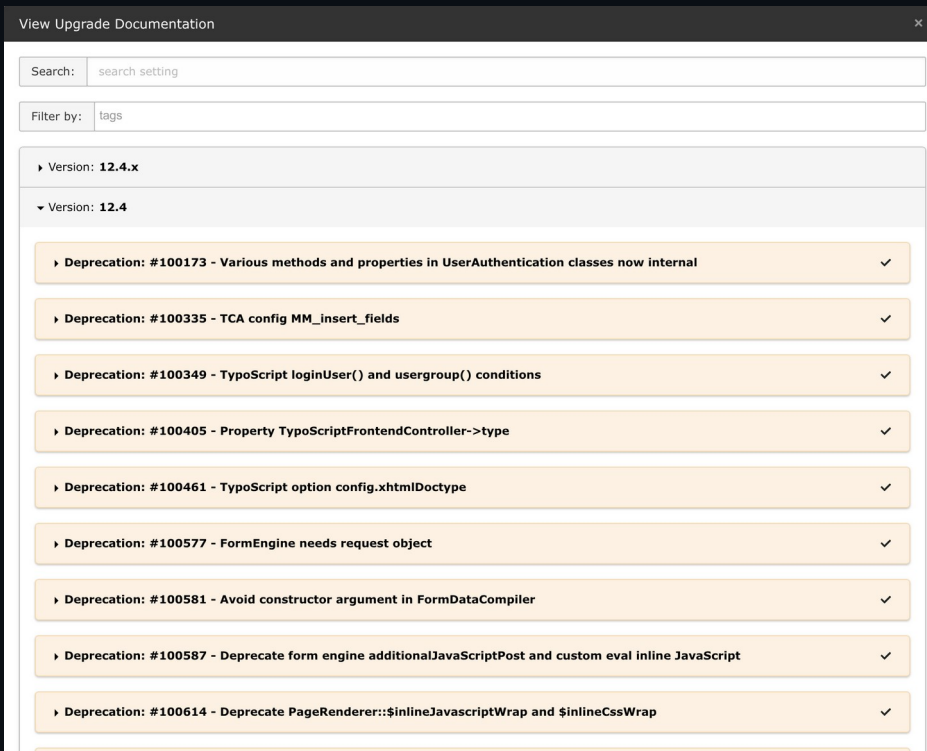
3 Post-Flight

Which **NEW** deprecations did I just inherit?

The version you just landed on deprecates things too. That list is the flight plan for the next upgrade.

```
Deprecation log · Extension Scanner · rector,  
next level set
```

The briefing is in your backend



Admin Tools > Upgrade > View Upgrade Documentation – TYP03 12.4.45

- ◆ Every changelog entry that ships with your core, searchable and filterable by tag.
- ◆ Mark entries as read so the list shrinks as you triage.
- ◆ You do not need the internet to do your NOTAM briefing.

Where to read them

Rendered documentation

`docs.typo3.org/c/typo3/cms-core/main/en-us/Index.html`

Filterable by version and type. The canonical source.

Shipped with the core

`vendor/typo3/cms-core/Documentation/Changelog/`

Offline, grep-able, and exactly matching your installed version.

In the Install Tool

Admin Tools > Upgrade > View Upgrade Documentation

Same entries, with read/unread state.

1

EVERYTHING BEFORE YOU TOUCH COMPOSER

Dispatch & Pre-Flight

The flight is won or lost before engine start.

How a crew starts the day

- ◆ Dispatch prepares a flight information package before the crew ever reaches the aircraft.
- ◆ Route, weather, NOTAMs, fuel, alternates, payload, and the MEL are already on paper.
- ◆ The crew verifies it – they do not invent it at the gate.
- ◆ Then they walk around the aircraft and look at it with their own eyes.

Nobody improvises a departure. The improvisation budget is saved for the things that actually go wrong.

Pre-Flight checklist

Challenge — Response. Read it out. Answer it honestly.

CURRENT VERSION	► known, patched, documented
CHANGELOGS	► read and triaged
EXTENSION SCAN	► run — strong matches at zero
THIRD-PARTY EXTENSIONS	► target-version ready
TESTS	► green and meaningful
ROLLBACK CRITERIA	► written down before departure

TARGET VERSION	► decided, dated, justified
DEPRECATIONS, CURRENT VERSION	► cleared or on the MEL
TCA MIGRATIONS	► checked — list is empty
PHP / DB / PLATFORM	► compatible and pinned
BACKUP AND RESTORE	► rehearsed, not assumed
TEAM	► briefed — shared mental map

Know your aircraft

You cannot plan a flight for an aircraft you have not inventoried.

```
# What am I actually running?
./vendor/bin/typo3 --version
composer show typo3/cms-core
php --version && composer show | wc -l

# What is installed, and what of it is mine?
./vendor/bin/typo3 extension:list
composer show --direct

# What is abandoned, outdated, or about to become someone else's problem?
composer outdated --direct
composer audit
```

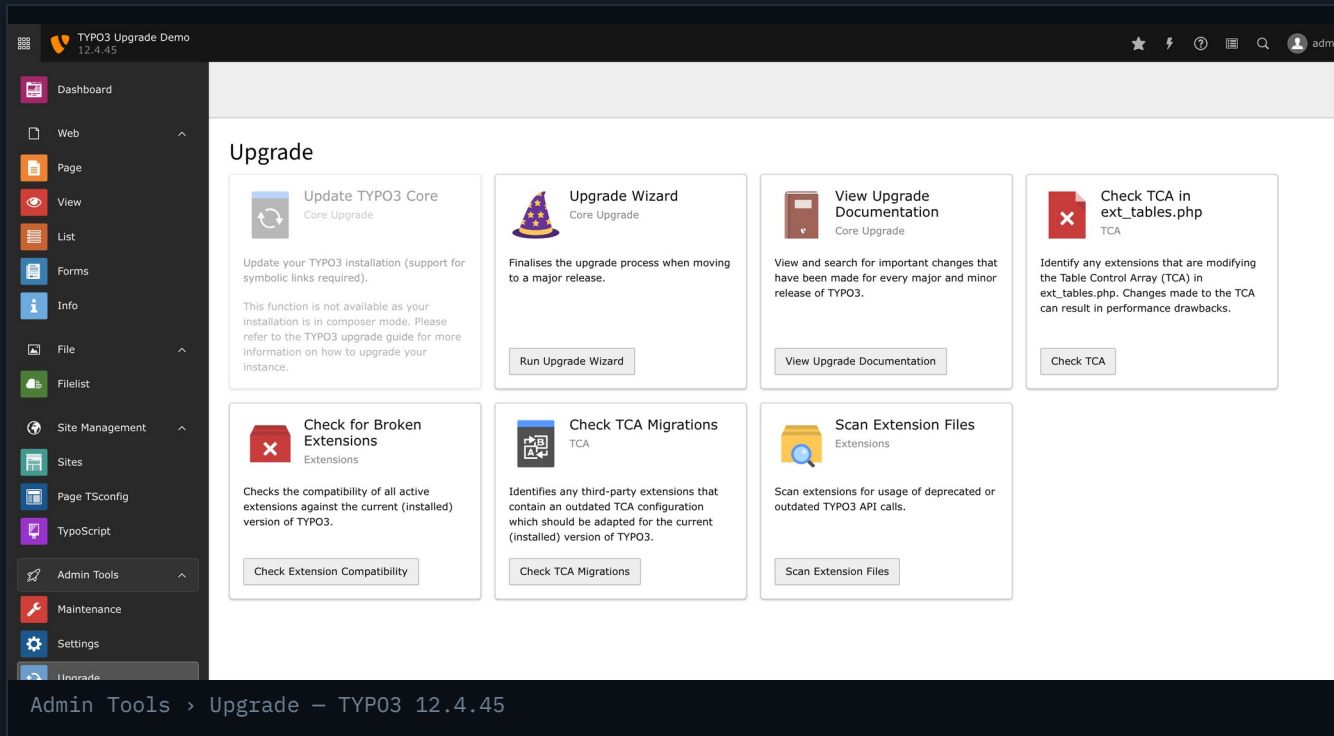
- ◆ Write the answers down. This is the aircraft you are flying, not the one you remember.
- ◆ Every extension is one of four things: maintained, adoptable, replaceable, or a reason not to depart.

You do not take a known-broken aircraft on a long leg.

Clear the deprecations of your CURRENT version first — while it still runs, still warns, and still forgives you.

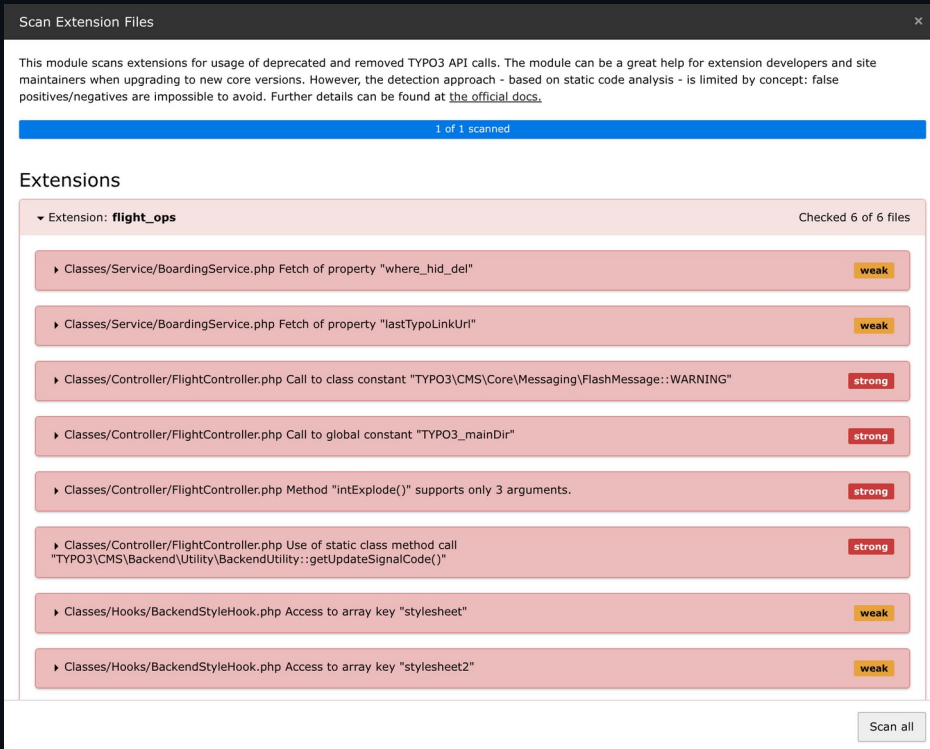
Every deprecation you leave behind is a removal you will meet mid-flight instead.

The Extension Scanner



- ◆ Ships with the core. Nothing to install, nothing to configure.
- ◆ Scans your extensions for deprecated and removed API from the version you are running.
- ◆ Also here: upgrade wizards, the changelog viewer, and the compatibility check.

What it tells you



Scan Extension Files – the demo extension flight_ops on TYPO3 12.4.45

- ◆ 13 findings in 6 files: 6 strong, 7 weak.
- ◆ Each finding names the file, the line, and the changelog entry behind it.
- ◆ This list is your work package – and your estimate.

Strong and weak matches

strong 6 findings

The scanner resolved the type and is certain. A definite usage of deprecated or removed API.

Fix it. This will break.

weak 7 findings

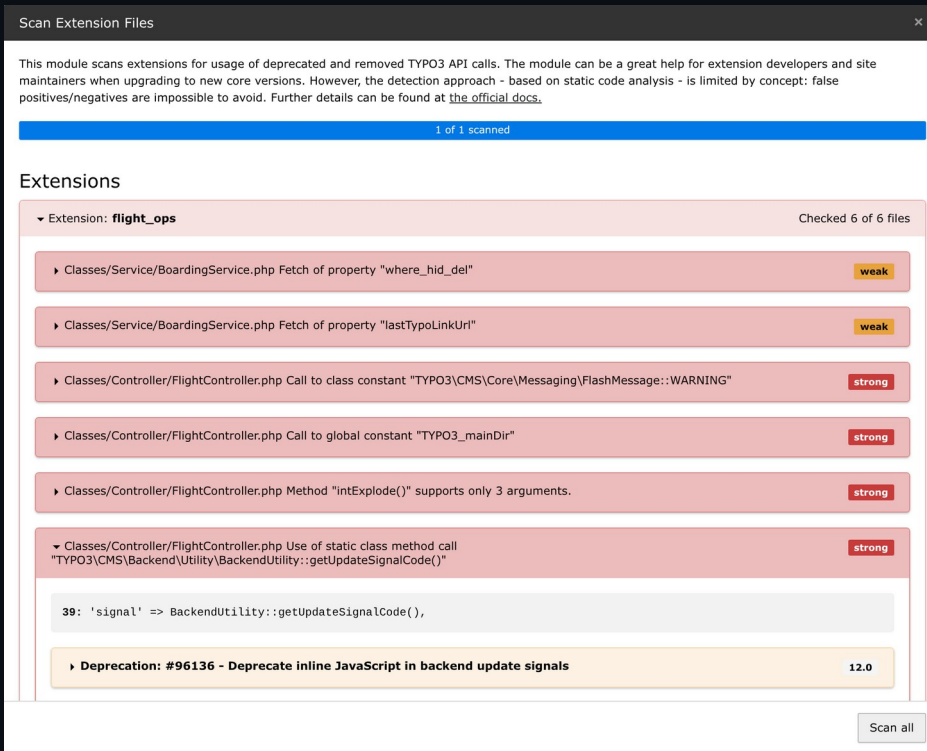
The name matches, but the type could not be resolved — \$obj->getTreeList() might be TYPO3's, or might be entirely yours.

Verify by hand. Then decide, and record the decision.

“The detection approach — based on static code analysis — is limited by concept: false positives/negatives are impossible to avoid.” — the Extension Scanner, about itself

A weak match is not noise. It is an item you have not decided about yet.

From finding to changelog in one click



A strong match, expanded – TYPO3 12.4.45

- ◆ The exact line: 39: 'signal' => BackendUtility::getUpdateSignalCode(),
- ◆ The exact entry: Deprecation #96136, shipped in 12.0.
- ◆ And in that entry: the migration you are supposed to apply.

Scanner → changelog → migration. That loop is the whole pre-flight method.

Where the scanner stops

It only reads PHP

TypoScript, Fluid, TCA, YAML, FlexForms and .htaccess are never looked at.

It cannot resolve dynamic code

String class names, variable method calls, and \$GLOBALS juggling stay invisible.

It does not have a rule for everything

QueryBuilder->execute() has been deprecated since 12.0 — and has no scanner rule at all.

It scans code, not behaviour

Changed defaults and changed rendering do not appear in any static scan.

This is not a flaw. It is one layer of defence — which is exactly why you add more.

The same scanner, on the command line

netresearch/extension-scanner-cli — the Install Tool check, in CI

```
$ ./vendor/bin/typo3 extension:scan flight_ops --no-progress
```

Results for: flight_ops

=====

File	Line	Type	Message	Indicator
Classes/Service/BoardingService.php	41	Property Protected	Fetch of property "where_hid_del"	WEAK
Classes/Service/BoardingService.php	36	Property Public	Fetch of property "lastTypoLinkUrl"	WEAK
Classes/Controller/FlightController.php	31	Class Constant	Call to class constant "FlashMes..."	STRONG
Classes/Controller/FlightController.php	40	Constant	Call to global constant "TYPO3_mainDir"	STRONG
Classes/Controller/FlightController.php	25	Method Argument Dropped Static	Method "intExplode()" supports only 3...	STRONG
Classes/Controller/FlightController.php	39	Method Call Static	Use of static call "getUpdateSignal..."	STRONG
Classes/Hooks/BackendStyleHook.php	17	Array Global	Access to array global "TBE_STYLES"	STRONG

[ERROR] Found 6 strong match(es) that WILL break on upgrade.

[WARNING] Found 7 weak match(es) that MAY need attention.

Total issues: 13 (6 strong, 7 weak)

- ◆ Reuses the matcher classes from EXT:install — the results are identical to the backend module.
- ◆ `—format=json | checkstyle, —all, —fail-on-weak`, and a non-zero exit code for CI.
- ◆ Not on Packagist today: install from the GitHub repository or the TER.

Make the aircraft stay clean

A scan you run once is a snapshot. A scan in CI is a standard.

```
# .gitlab-ci.yml
extension-scanner:
  stage: qa
  script:
    - composer install --no-progress
    - ./vendor/bin/typo3 extension:scan --all --no-progress
  # exits non-zero on strong matches -> the pipeline fails

# Checkstyle output for the merge-request annotations
- ./vendor/bin/typo3 extension:scan --all --format=checkstyle > scan.xml
artifacts:
  reports:
    codequality: scan.xml
```

- ◆ New deprecated usage never gets merged in the first place.
- ◆ The next upgrade then starts from zero instead of from archaeology.

TCA is migrated behind your back

- ◆ On every single request, TYP03 rewrites outdated TCA in memory before anything uses it.
- ◆ Your old configuration keeps working — so nothing ever tells you it is old.
- ◆ `TcaFactory::migrateTca()` raises a single `E_USER_DEPRECATED` carrying the full list of what it changed.
- ◆ In production that goes into the deprecation log, if you enabled it. Usually nobody looks.

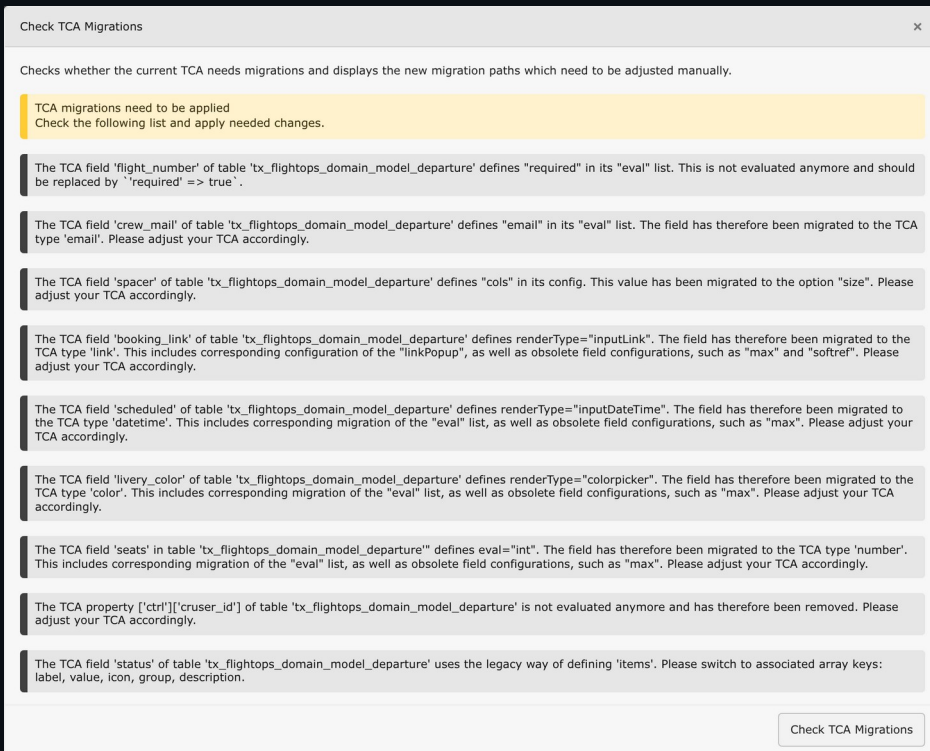
This is the autopilot quietly trimming out an imbalance. The aircraft flies straight — right up until the day the autopilot is removed.

“...these migrations will be removed.”

The core says it out loud, on every request, in the deprecation it raises. An auto-migration is not a fix. It is an Airworthiness Directive with an open deadline.

Verbatim from TcaFactory: “Automatic TCA migration done during bootstrap. Please adapt TCA accordingly, these migrations will be removed. The backend module ‘Configuration → TCA’ shows the modified values.”

The tool nobody opens



Admin Tools › Upgrade › Check TCA Migrations – TYP03 13.4.34

- ◆ Nine findings from one small demo extension – required, email, cols, renderTypes, items, cruser_id.
- ◆ The Extension Scanner never reports these. Rector does not fix them. Fractor does not read TCA.
- ◆ Its sibling card, Check TCA in ext_tables.php, finds TCA built in the wrong place.

There is no CLI command for this one. It is backend-only – which is exactly why it gets skipped.

Your functional tests already knew

typo3/testing-framework ships failOnDeprecation="true"

```
$ vendor/bin/phpunit -c Build-FunctionalTests.xml
```

```
PHPUnit 11.5.56 by Sebastian Bergmann and contributors.
```

```
D                                     1 / 1 (100%)
```

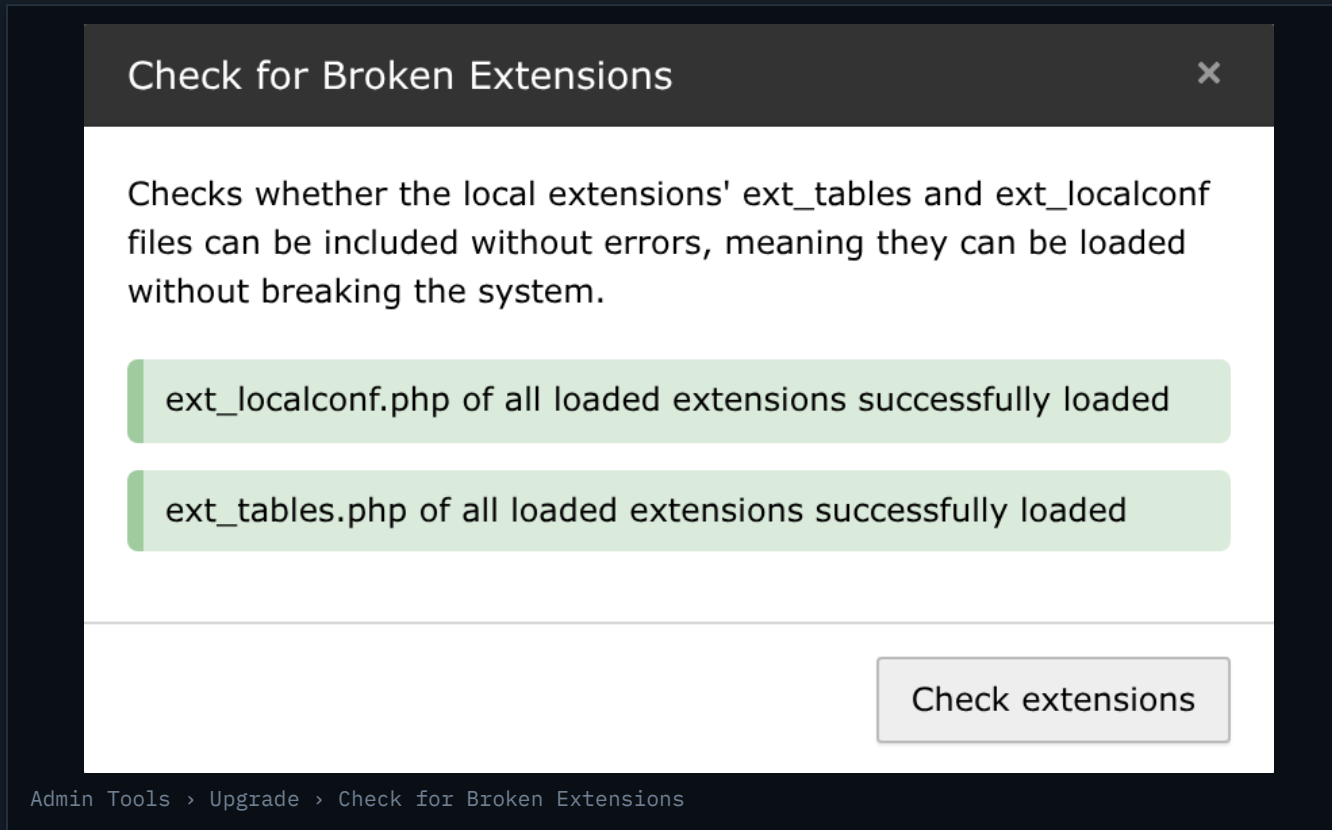
```
1 test triggered 1 deprecation:
```

```
1) .../cms-core/Classes/Configuration/Tca/TcaFactory.php:176
Automatic TCA migration done during bootstrap. These migrations will
be removed. Please adapt these areas:
The TCA field 'flight_number' defines "required" in its "eval" list.
... 9 more messages
OK, but there were issues! Tests: 1, Assertions: 1, Deprecations: 1.
```

- ◆ The assertion passed. The suite still failed — exit code 1, because one deprecation was raised.
- ◆ This is why CI turns red after a core bump when you changed nothing at all.
- ◆ Do not silence it. That red build is the only thing that noticed.

FlexFormTools does the same for FlexForm data structures — same deprecation, same red build.

Does it even load?



◆ Includes every `ext_localconf.php` and `ext_tables.php` and reports what explodes.

◆ The cheapest possible check – and the first one worth running after any bump.

Green means loadable. It does not mean correct. Nothing here says your site works.

Rector as reconnaissance

—dry-run changes nothing and tells you the size of the job

```
$ ./vendor/bin/rector process --dry-run
```

```
4 files with changes
```

```
=====
```

```
1) packages/flight_ops/Classes/Controller/FlightController.php:27
```

```
-         FlashMessage::WARNING,  
+         \TYPO3\CMS\Core\Type\ContextualFeedbackSeverity::WARNING,
```

```
Applied rules:
```

```
* RenameClassConstFetchRector
```

```
3) packages/flight_ops/Classes/Service/BoardingService.php:20
```

```
-         ->select('*')  
-         ->from('tx_flightops_domain_model_departure')  
-         ->execute()  
+         ->select('*')->from('tx_flightops_domain_model_departure')->executeQuery()
```

```
Applied rules:
```

```
* MigrateContentObjectRendererLastTypoLinkPropertiesRector
```

```
* MigrateQueryBuilderExecuteRector
```

```
[OK] 4 files would have been changed (dry-run) by Rector
```

- ◆ ssch/typo3-rector, configured with Typo3LevelSetList::UP_TO_TYPO3_13.
- ◆ It found `->execute()`— the deprecation the Extension Scanner has no rule for.
- ◆ In phase 1 you only look. You do not let it write yet.

Fractor covers the other half

Everything that is not PHP – and nothing else looks at it

```
$ ./vendor/bin/fractor process --dry-run
```

```
1 file with changes
```

```
=====
```

```
1) packages/flight_ops/Configuration/TypoScript/setup.typoscript:1
```

```
@@ Line 1 @@
```

```
-<INCLUDE_TYPOSCRIPT: source="FILE:EXT:flight_ops/.../Board.typoscript">
```

```
-<INCLUDE_TYPOSCRIPT: source="DIR:EXT:flight_ops/.../Partials" extensions="typoscript">
```

```
+@import 'EXT:flight_ops/Configuration/TypoScript/Plugin/Board.typoscript'
```

```
+@import 'EXT:flight_ops/Configuration/TypoScript/Partials/*.typoscript'
```

```
@@ Line 12 @@
```

```
-[globalVar = TSFE:id = 42]
```

```
+[traverse(page, "uid") == 42]
```

```
-[globalVar = GP:type = 98]
```

```
+[request && traverse(request.getQueryParams(), 'type') == 98]
```

```
Applied rules:
```

```
★ MigrateLegacyTypoScriptConditionsFractor
```

```
★ MigrateTypoScriptPageConditionToTraverseFractor
```

```
★ MigrateIncludeTypoScriptSyntaxFractor
```

```
[OK] 1 file would have been changed (dry-run) by Fractor
```

- ◆ a9f/typo3-fractor: TypoScript, Fluid, YAML, XML, XLIFF, FlexForm, .htaccess, composer.json.
- ◆ The Extension Scanner never opened this file. Rector never opened this file.

Four tools, four blind spots

Real numbers from the demo extension in this talk

10

FOUND ONLY BY THIS TOOL

Extension Scanner

13 issues, 10 of them reported by no other tool

2

FOUND ONLY BY THIS TOOL

Rector

->execute() and clearCacheOnLoad –
neither of them has a scanner rule

1

FOUND ONLY BY THIS TOOL

Fractor

The TypeScript file none of the others
ever opens

9

FOUND ONLY BY THIS TOOL

TCA migration check

9 TCA messages – no other tool here
reports a single one of them

Only 3 of the scanner's 13 findings are auto-fixed by Rector.

No single tool clears the aircraft. That is the whole point of layered defences – the holes in the slices do not line up.

Can the fleet even fly?

Every third-party extension is a go / no-go item

```
$ composer why-not typo3/cms-core 13.4
```

```
typo3/cms-base-distribution - requires typo3/cms-core (^12.4)
typo3/cms-backend          v12.4.45 requires typo3/cms-core (12.4.45)
typo3/cms-extbase          v12.4.45 requires typo3/cms-core (12.4.45)
...
webvision/flight-ops       dev-main requires typo3/cms-core (^12.4)
typo3/cms-core             v13.4.0 requires doctrine/dbal (^4.2.1)
typo3/cms-base-distribution - does not require doctrine/dbal (but 3.10.6 is installed)
```

- ◆ For every blocker: is there a compatible release, a fork, a replacement – or is it yours to adopt?
- ◆ An abandoned extension is not a technical problem. It is a decision, and it belongs in phase 1.

Your own tooling can ground you

A real failure from building this talk

```
$ composer require typo3/cms-core:^13.4 ... -W
```

Your requirements could not be resolved to an installable set of packages.

Problem 1

- Root composer.json requires typo3/cms-install ^13.4
-> satisfiable by typo3/cms-install[v13.4.0, ..., v13.4.34].
- typo3/cms-install[v13.4.10, ..., v13.4.34] require nikic/php-parser ^5.4.0
-> found nikic/php-parser[v5.4.0, ..., v5.8.0] but these were not loaded,
likely because it conflicts with another require.

Installation failed, reverting ./composer.json and ./composer.lock

- ◆ The blocker was not the project. It was rector, pinning nikic/php-parser in require-dev.
- ◆ Dev tools share your dependency graph – until you isolate them.
- ◆ Fix: update the tools in the same transaction, or install them in their own tools/ directory.

The MEL – what you accept as broken

- ◆ Aviation has a legal list of equipment that may be inoperative while the aircraft still dispatches.
- ◆ Every item carries a condition, a procedure, and a rectification deadline.
- ◆ You will not fix every finding before departure. That is normal and professional.
- ◆ What is not professional is not writing down which ones you skipped, and why.

An undocumented accepted finding is not accepted. It is forgotten – and it will be rediscovered at the worst possible moment.

Tests are your instruments

- ◆ Clicking through the site is VFR – fine in daylight, useless at night and in cloud.
- ◆ A functional test boots an actual TYPO3 and runs your code against the real core.
- ◆ That is the difference: every other tool in this talk READS your code. Tests RUN it.
- ◆ No tests? Then a smoke checklist before you depart – not after it breaks.

Instruments do not stop turbulence. They tell you which way is up while it happens.

Every line your tests execute is a line the core can warn you about.

Deprecations are raised at runtime. Removed API fails at runtime. A line of code no test ever executes is a line no tool can warn you about — until an editor executes it in production.

This is why coverage is not a code-quality nicety during an upgrade. It is the difference between finding things in week one and finding them on the night.

Reading the code vs running it

Static analysis — reads

- ◆ Extension Scanner, Rector, Fractor, PHPStan
- ◆ Limited to a fixed list of known patterns
- ◆ Cannot resolve dynamic calls or string class names
- ◆ Never sees your TCA, TypoScript or data as the core sees them
- ◆ Fast, cheap, runs on every commit

Tests — run

- ◆ The real core, the real bootstrap, a real database
- ◆ Whatever your code actually does, including the dynamic parts
- ◆ Catches changed defaults and changed rendering
- ◆ Reports the core's own deprecation text, verbatim
- ◆ Slower, and the only layer that executes anything

You need both. Static analysis narrows the search; tests prove the result.

Cover what touches the core

Services and repositories

Anything calling QueryBuilder, FAL, links, language or context. This is where removed API hides.

ViewHelpers and templates

Fluid changes quietly. A render test costs minutes and catches silent output changes.

TCA and FlexForms

A test that only boots is already enough — the core reports its own migrations for you.

Plugins and controllers

One functional request per plugin. Registration, routing and TypoScript all change between majors — and a plugin that stops rendering is silent.

Event listeners, hooks and middleware

Registration APIs move. A test proves yours is still called at all.

Commands and scheduler tasks

Nobody clicks these during a smoke test. They break in the night, unattended.

Rule of thumb: if it calls TYPO3 API, one functional test that executes it pays for itself at every single upgrade — forever.

What each kind of problem looks like in a test run

Breaking change

- ◆ The API is gone — the code cannot run at all
- ◆ PHPUnit reports an ERROR, with file and line
- ◆ Call to undefined method QueryBuilder::execute()
- ◆ Urgency: now. It cannot run on the new version.

Deprecation

- ◆ Still works — the core raises E_USER_DEPRECATED
- ◆ failOnDeprecation turns it into a build failure
- ◆ The message contains the migration instruction
- ◆ Urgency: before the next major. On your schedule.

One test run separates “must fix to boot” from “must fix before the next flight” — and hands you the migration text for both.

Write the QRH

Quick Reference Handbook – the prepared answer to the expected failure

WHEN THIS HAPPENS

DO THIS

REFERENCE

Class not found: ObjectManager

Removed in v12 – inject dependencies via constructor

Breaking #96107

Call to undefined method QueryBuilder::execute()

executeQuery() / executeStatement()

Deprecation #96972

FlashMessage severity type error

ContextualFeedbackSeverity enum

Deprecation #97787

TypeScript condition never matches

[loginUser()] / [usergroup()] gone – use frontend.user /
backend.user

Deprecation #100349

An error you predicted is a checklist item. The same error unpredicted is an emergency. Same error – different day.

The departure briefing

- ◆ Say the plan out loud to the people who will fly it with you.
- ◆ Who runs which step, in which order, and who watches what.
- ◆ What going wrong looks like – concretely, not as a feeling.
- ◆ Who is reachable, and who is explicitly not on duty.

A shared mental map is the reason two pilots are better than one. Two people with different plans are worse than one person with a plan.

Go / No-Go

GO

- ✓ Strong matches at zero, or on the MEL with a date
- ✓ All third-party extensions resolved
- ✓ Backup restored successfully at least once
- ✓ Rollback criteria written and understood
- ✓ The window is long enough for the plan plus surprises

NO-GO

- ✗ "We will find out during the upgrade"
- ✗ An unowned extension nobody can update
- ✗ A backup that has never been restored
- ✗ A window that fits only the happy path
- ✗ The person who knows the system is on holiday

No-Go is a valid, professional outcome. Delaying a flight costs money. The alternative costs more.

2

THE UPGRADE WINDOW ITSELF

Take-Off & Cruise

Now you fly the plan you wrote. You do not write a new one.

Sterile cockpit

- ◆ Below 10,000 feet, crews may only discuss what is necessary for the flight.
- ◆ No small talk, no paperwork, no non-essential radio calls. It is a rule, not a mood.
- ◆ Your equivalent: content freeze, code freeze, deploy freeze — announced in advance.
- ◆ “While you are in there anyway, could you also...” — no. That is a separate flight.

Most upgrade disasters are not caused by the upgrade. They are caused by the third thing someone did during it.

Upgrade checklist

Challenge — Response

FREEZE ANNOUNCED	▶ content, code, deploys
BACKUP TAKEN	▶ database and files, verified
PACKAGES UPGRADED	▶ core and extensions, one transaction
FRACTOR RUN	▶ AFTER packages – reviewed hunk by hunk
TCA MIGRATIONS	▶ re-checked on the new core
WIZARDS RUN	▶ listed, executed, verified
SMOKE TEST	▶ frontend and backend, by a human

BRANCH CREATED	▶ everything under version control
PLATFORM RAISED	▶ PHP and composer platform config
RECTOR RUN	▶ AFTER packages – reviewed hunk by hunk
EXTENSION SCAN	▶ re-run with the TARGET version's rules
SCHEMA UPDATED	▶ database compare, deliberately
CACHES FLUSHED	▶ and warmed

The order is not a preference

Do these in this order, or do them twice

- | | | |
|---|-----------------|---|
| 1 | Branch | git checkout -b upgrade/v13 — everything after this is reversible |
| 2 | Platform | PHP version, composer config platform.php, extension constraints |
| 3 | Packages | composer update -W — the core and all extensions move together |
| 4 | Rector | Now the new rule sets and the new code are both on disk |
| 5 | Fractor | TypoScript, Fluid, YAML — the half Rector never sees |
| 6 | Scan | Extension Scanner AND the TCA migration check — both now carry the target rules |
| 7 | Schema | extension:setup and the database compare — the point of no return |
| 8 | Wizards | upgrade:list, then upgrade:run — repeatable, scriptable |
| 9 | Caches | flush, warm, and only then look at the site |

Steps 4 and 5 run AFTER step 3 — that is the one everybody gets wrong.

Rector before the package upgrade refactors against the old world.

The rule sets ship with the tool. The code they must match ships with the new core. You need both on disk before you let anything rewrite your files.

Same for Fractor. Bump first, refactor second, review always.

The bump

One transaction – the core and everything that depends on it

```
$ composer update -W
```

```
Loading composer repositories with package information
Updating dependencies
Lock file operations: 12 installs, 47 updates, 6 removals
- Upgrading typo3/cms-core (v12.4.45 => v13.4.34)
- Upgrading typo3/cms-backend (v12.4.45 => v13.4.34)
- Upgrading typo3/cms-extbase (v12.4.45 => v13.4.34)
- Upgrading doctrine/dbal (3.10.6 => 4.4.1)
...
Writing lock file
Generating autoload files
Publishing public assets of TYPO3 extensions

No security vulnerability advisories found.
```

- ◆ Compare that last line with the one from phase 1. That is what leaving ELTS looks like.
- ◆ If it refuses: composer why-not, then fix the real blocker – do not reach for –ignore-platform-reqs.

V1

On the take-off roll there is a speed after which you are going flying, whatever happens. Your V1 is the database schema migration.

Before it: a branch and a lock file. After it: a restore from backup. Know exactly where your V1 is — and say it out loud before you cross it.

Now let Rector write

Same command, without `—dry-run`

```
$ ./vendor/bin/rector process
```

```
4 files with changes
```

```
=====
```

```
1) packages/flight_ops/Classes/Controller/FlightController.php:27  
   * RenameClassConstFetchRector
```

```
2) packages/flight_ops/Classes/Hooks/BackendStyleHook.php:19  
   * MigrateTypoScriptFrontendControllerTypeRector
```

```
3) packages/flight_ops/Classes/Service/BoardingService.php:20  
   * MigrateQueryBuilderExecuteRector  
   * MigrateContentObjectRendererLastTypoLinkPropertiesRector
```

```
4) packages/flight_ops/ext_emconf.php:5  
   * ExtEmConfRector
```

```
[OK] 4 files have been changed by Rector
```

- ◆ ExtEmConfRector removed `clearCacheOnLoad` — a 12.1 deprecation with no scanner rule either.
- ◆ Two of these five rules fixed things the Extension Scanner never reported.

Rector is a co-pilot, not an autopilot

- ◆ Read every hunk. It rewrote your code — you are still the one who signs for it.
- ◆ Commit in slices, one rule set at a time, so a bad change is one revert and not an archaeological dig.
- ◆ It produces unformatted output — run your CGL fixer afterwards.
- ◆ Never on production. Never outside version control. The tool's own docs say so.

A co-pilot flies the aircraft. A co-pilot does not get to decide where you are going.

The fix that broke the plugin

Rector migrated the plugin registration. Correctly. And the site went blank.

What Rector did

- ◆ Moved the plugin from a `list_type` subtype to a real `CType`
- ◆ Exactly what TYP03 v13 asks you to do
- ◆ And it scaffolded the upgrade wizard for the data migration
- ◆ ...with a `// TODO: Add this mapping yourself!` left in it

What nobody was told

- ◆ Every existing `tt_content` row still says `CType=list`
- ◆ The plugin now matches nothing and renders nothing
- ◆ No error, no log entry, no scanner finding
- ◆ An editor reports it three weeks later

Code migration and data migration are two different jobs. Refactoring tools do the first one and, at best, hand you a stub for the second.

One test noticed. It was the one that renders the plugin.

TypeScript object path `"tt_content.list.20.flightops_board"` does not exist. Not a deprecation, not a scanner match, not a Rector diff — a failing assertion in the only test that executes the whole chain.

This is why “cover your plugins” is on the pre-flight checklist. A unit test on the service would have stayed green all the way to production.

Then Fractor, for everything else

```
$ ./vendor/bin/fractor process

1 file with changes
=====

1) packages/flight_ops/Configuration/TypoScript/setup.typoscript:1

* MigrateIncludeTypoScriptSyntaxFractor
* MigrateLegacyTypoScriptConditionsFractor
* MigrateTypoScriptPageConditionToTraverseFractor

[OK] 1 file has been changed by Fractor
```

- ◆ A silently non-matching TypoScript condition does not raise an error. It just quietly does nothing.
- ◆ Those are the bugs that get reported three weeks later by an editor, not by your logs.

Now the upgrade is a list of red tests.

Run the suite. Take the top failure. Fix it. Run again. You always know how many are left and how far you have come.

Without coverage the same work is: change something, click around the site, hope you thought of everything, repeat. That is not a procedure — that is a mood.

What a breaking change looks like

The same suite, the same code — five minutes after the composer bump

```
$ Build/Scripts/runTests.sh -s functional
```

There were 6 errors:

```
1) FlightBoardTest::boardRendersFlightNumbersFromTheDatabase
Error: Undefined constant TYPO3\CMS\Core\Messaging\FlashMessage::WARNING
packages/flight_ops/Classes/Controller/FlightController.php:31
```

```
4) BoardingServiceTest::findDeparturesReturnsAllRows
Error: Call to undefined method
    TYPO3\CMS\Core\Database\Query\QueryBuilder::execute()
packages/flight_ops/Classes/Service/BoardingService.php:26
```

ERRORS!

Tests: 8, Assertions: 2, Errors: 6, Deprecations: 4.

- ◆ File, line, and the exact method. No bisecting, no guessing, no clicking.
- ◆ Six failures, two root causes — the count tells you the size, not just the fact.
- ◆ The Extension Scanner has no rule for this method. The suite did not need one.

Deprecation on Monday. Fatal error on Tuesday. Same test, same line.

On 12.4 that suite reported 4 deprecations and passed every assertion. On 13.4 the same suite reports 6 errors. Nothing about the code changed — only the core underneath it.

This is the whole talk in one command. Your tests are where “today's deprecation is tomorrow's breaking change” stops being a slogan and becomes a line number.

Working the failures

- ◆ Errors first: a removed API means the code cannot even run. That is your blocking set.
- ◆ Deprecations second: they still work, so they are scheduled work, not emergency work.
- ◆ One fix, one run, one commit — so a wrong fix is one revert, not an investigation.
- ◆ A failure you cannot explain is a changelog you have not read yet.

The suite also tells you when to stop. “Green” is a fact. “I think it works now” is not.

Four states of one pipeline

Every number measured on the workshop project — same 8 tests throughout

12.4 — before departure

Everything passes. The core names four things v13 will change.

8 tests · 11 assertions · 4 deprecations

1

EXIT

13.4 — straight after the bump

execute() and FlashMessage::WARNING are gone. Six tests, two root causes.

8 tests · 2 assertions · 6 errors

2

EXIT

13.4 — after Rector and Fractor

Half cleared — and Rector's own fix broke the plugin. See next slide.

8 tests · 6 assertions · 3 errors

2

EXIT

13.4 — after the hand work

A removed constant, one TCA rule, and a data migration later.

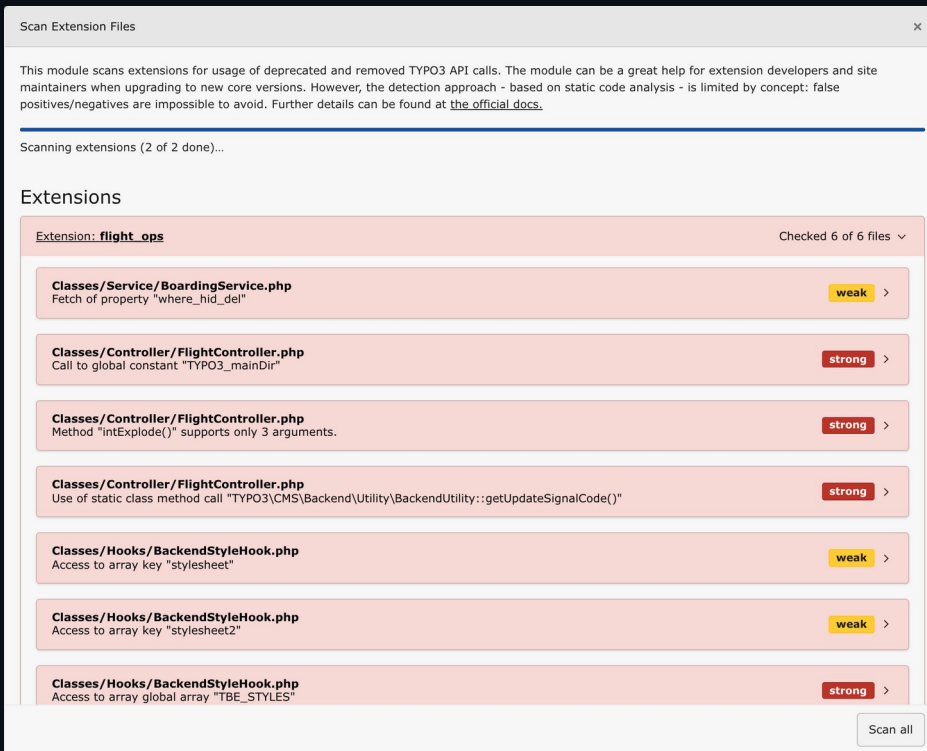
OK — 8 tests · 11 assertions

0

EXIT

You are never guessing how far along you are. That is what coverage buys you on the night of the upgrade.

Scan again – same tool, new truth



Scan Extension Files – the same extension, now on TYPO3 13.4.34

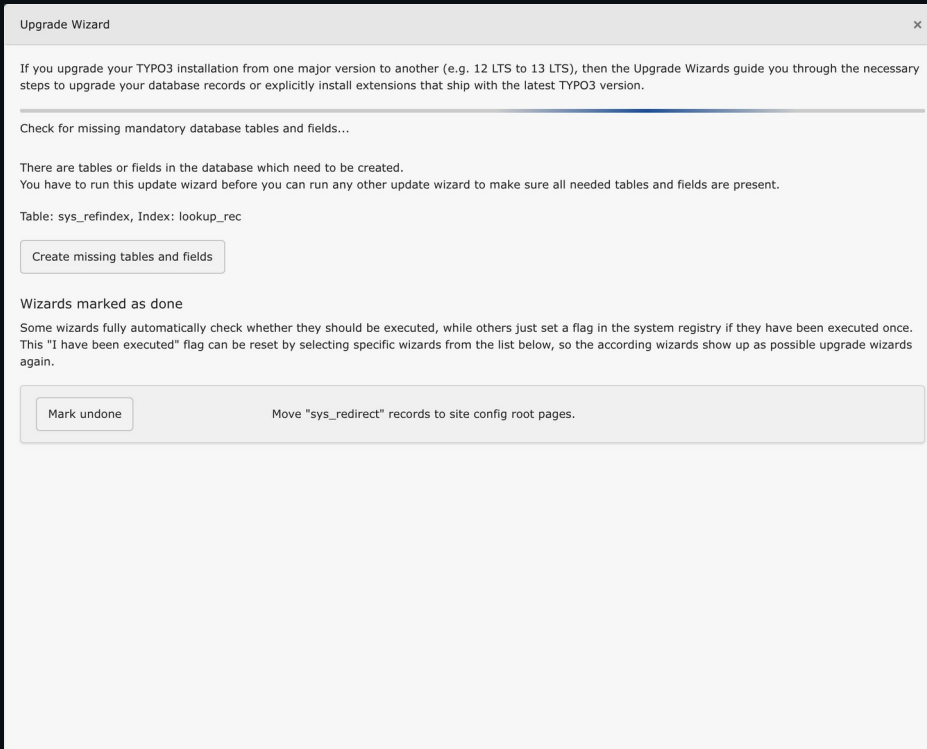
◆ 13 findings before, 11 after – Rector cleared three, and the scanner found one that is new.

◆ New and strong: TypoScriptFrontendController itself, Deprecation #105230, a v13 entry the v12 scanner could not possibly have known.

◆ Same tool, same code, new truth – because the rule set ships with the core.

That new finding is not a phase-2 problem. It is the first line of your phase-3 backlog.

Upgrade wizards



Admin Tools › Upgrade › Upgrade Wizard — TYPO3 13.4.34

- ◆ Wizards migrate DATA and CONFIGURATION — the part no refactoring tool can touch.
- ◆ Schema first: missing tables and fields must exist before any wizard may run.
- ◆ A wizard only appears when it actually has something to migrate — an empty list is an answer.
- ◆ “Mark undone” resets the flag, so a wizard can be re-run on the next environment.

This is also why you rehearse on a copy of production data. On an empty database every wizard looks finished.

Make it repeatable

Anything you do by hand once, you will do wrong the second time

```
# What is waiting, and what already ran?
./vendor/bin/typo3 upgrade:list
./vendor/bin/typo3 upgrade:list --all

# Run one, or run them all - scriptable, repeatable, reviewable
./vendor/bin/typo3 upgrade:run pagesRecyclerDoktypeMigration
./vendor/bin/typo3 upgrade:run

# Schema and extension state
./vendor/bin/typo3 extension:setup
./vendor/bin/typo3 database:updateschema
./vendor/bin/typo3 cache:flush
```

- ◆ The same script runs on your machine, on staging, and in production.
- ◆ That is how the production run stops being the first run.

When it breaks: Aviate · Navigate · Communicate

STEP 1

Aviate

Keep the system in a known state. Stop changing things. Do not panic-edit production. The aircraft is still flying.

STEP 2

Navigate

Find out where you actually are. Which step? What does the log say? Is this in your QRH? Read the actual error, not the summary.

STEP 3

Communicate

Team, community, Slack, documentation, the changelog. Asking for help early is a procedure, not a confession.

The order matters. People who communicate before they aviate crash while typing.

Two kinds of error

You predicted this one

- ◆ It is in your QRH with a prepared response
- ◆ The changelog entry is already open in a tab
- ◆ You apply the migration and continue
- ◆ Elapsed time: minutes. Stress: low.

You did not

- ◆ Stay calm — this is normal, not a failure
- ◆ Re-read the changelogs; you probably skipped one
- ◆ Check the extension's own upgrade instructions
- ◆ Ask. Then write it into the QRH for next time.

Preparation does not remove surprises. It shrinks the set of them, and it buys you the calm to handle what is left.

The go-around

- ◆ If the approach is not stable, you go around. Every time. It is not a failure – it is a manoeuvre.
- ◆ Pilots practise go-arounds. The decision is pre-briefed, so it costs nothing to make.
- ◆ Your rollback criteria were written in phase 1, when you were calm and nobody was watching.
- ◆ Execute them without ego. “We are already three hours in” is not a rollback criterion.

The reason to define abort criteria before departure is that you are the worst possible judge of them at 02:00 with a broken site.

3

THE PHASE EVERYBODY SKIPS

Approach, Landing & Debrief

The flight does not end at touchdown. It ends at the gate, with the paperwork done.

Post-Flight checklist

Challenge — Response

FRONTEND	▶ smoke-tested by a human
SCHEDULER	▶ tasks registered and running
EXTENSION SCAN	▶ re-run — new deprecations recorded
FERRY EQUIPMENT	▶ shims and upgrade-only packages removed
DOCUMENTATION	▶ updated while you still remember
NEXT FLIGHT	▶ planned from what you just learned

BACKEND	▶ editors can do their actual daily work
DEPRECATION LOG	▶ enabled and read
TCA MIGRATIONS	▶ cleared, or in the backlog
CONSTRAINTS	▶ re-pinned to the new reality
DEBRIEF	▶ held — predicted vs actual

Green again is not the end of the work. It is the end of the guessing.

A green pipeline on the new version means every path your tests cover behaves. It says nothing about the paths they do not cover — so the smoke test still happens, by a human, on purpose.

And the honest measurement for next time: which failures did your tests catch, and which did a person find afterwards? That gap is your coverage backlog.

Smoke-test the whole aircraft

- ◆ Frontend: every page type, every plugin, every language, images, forms, search.
- ◆ Backend: the tasks your editors actually perform — not the ones you find interesting.
- ◆ The boring infrastructure: mail, redirects, scheduler, indexing, caches, CDN.
- ◆ Have a person who did not do the upgrade click through it. They see what you cannot.

Your test suite proves the code runs. A human proves the site works.

First look after landing

The screenshot shows the TYPO3 Upgrade Demo interface. The header displays 'TYPO3 Upgrade Demo 13.4.34' and a user profile 'admin'. The left sidebar contains navigation links: Dashboard, Web, Page, View, List, Forms, Info, File, Filelist, Site Management, Sites, Settings, Page TSconfig, TypoScript, Admin Tools, and Maintenance. The main content area is titled 'Upgrade' and contains seven task cards:

- Update TYPO3 Core** (Core Upgrade): Update your TYPO3 installation (support for symbolic links required). This function is not available as your installation is in composer mode. Please refer to the TYPO3 upgrade guide for more information on how to upgrade your instance. Button: Update TYPO3 Core.
- Upgrade Wizard** (Core Upgrade): Finalises the upgrade process when moving to a major release. Button: Run Upgrade Wizard...
- View Upgrade Documentation** (Core Upgrade): View and search for important changes that have been made for every major and minor release of TYPO3. Button: View Upgrade Documentation...
- Check TCA in ext_tables.php** (TCA): Identify extensions that are modifying the Table Configuration Array (TCA) in `ext_tables.php`. Button: Check TCA...
- Check for Broken Extensions** (Extensions): Checks the compatibility of all active extensions against the current (installed) version of TYPO3. Button: Check Extension Compatibility...
- Check TCA Migrations** (TCA): Identifies any third-party extensions that contain an outdated TCA configuration which should be adapted for the current (installed) version of TYPO3. Button: Check TCA Migrations...
- Scan Extension Files** (Extensions): Scan extensions for usage of deprecated or outdated TYPO3 API calls. Button: Scan Extension Files...

The same instance, now TYPO3 13.4.34 – the demo for this talk, upgraded live

- ◆ The version in the header is the only part of the upgrade that was ever guaranteed.
- ◆ Everything else on this checklist is what makes it true for your editors too.

TCA after landing

New migrations arrived

- ◆ The new core added migrations your old core never had
- ◆ Run Check TCA Migrations again — the list is not the one you cleared
- ◆ Every new entry is a fresh deprecation you just inherited

Old migrations left

- ◆ A migration you were silently relying on may be gone
- ◆ Then it is not a deprecation any more — the config is simply ignored
- ◆ Nothing errors. The field just renders wrong, or not at all

This is the half that bites without a stack trace: the autopilot was removed, and the aircraft is quietly out of trim. Only the checklist catches it.

The deprecation log is your tech log

Pilots write up every defect after the flight, even the small ones

```
# config/system/additional.php - switch it on, then actually read it
$GLOBALS['TYPO3_CONF_VARS']['LOG']['TYPO3']['CMS']['deprecations']['writerConfiguration'] = [
    \Psr\Log\LogLevel::NOTICE => [
        \TYPO3\CMS\Core\Log\Writer\FileWriter::class => [
            'disabled' => false,
        ],
    ],
];

# then, after a day of real traffic and real editing:
tail -f var/log/typo3_deprecations_*.log
```

- ◆ Static analysis reads code paths. The deprecation log reports the ones actually taken.
- ◆ It catches the dynamic calls the Extension Scanner structurally cannot see.
- ◆ Let it run under real editor traffic for a few days. That is where the rest hides.

Scan again. You are not finished — you are at the start of the next one.

The version you just landed on deprecates things too. That list is not a leftover. It is the flight plan for the next upgrade, handed to you for free.

Run it now, while the context is still in your head, and put the findings in the backlog with the changelog links attached.

Remove the ferry equipment

- ◆ Compatibility shims and polyfills you added “just for the migration”.
- ◆ Upgrade-only dev packages — or move them into an isolated tools/ install.
- ◆ The @todo v13 comments you actually resolved. Delete them; do not let them rot.
- ◆ Version constraints still pointing at the old world in your own extensions.

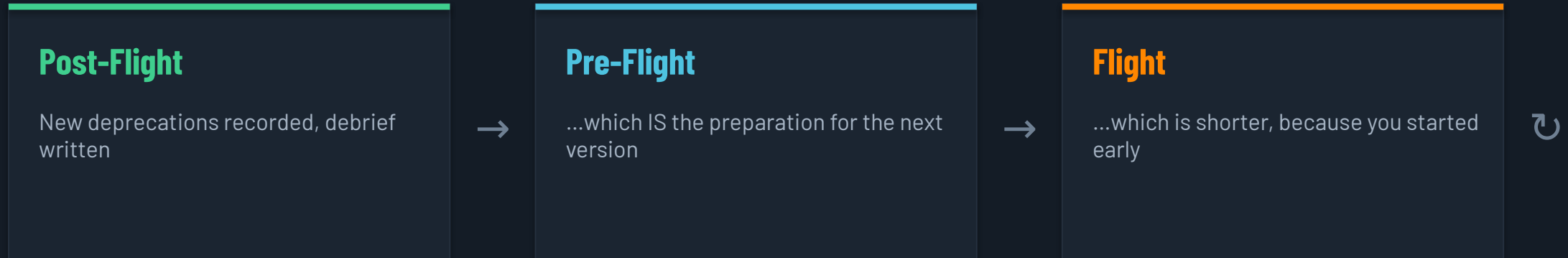
An aircraft flies home with the ferry tank still fitted exactly once. After that it is just weight you pay for on every leg.

The debrief

- ◆ What did we predict correctly? Keep it in the checklist – it earned its place.
- ◆ What surprised us? That is a new QRH entry, written now, not “soon”.
- ◆ What took longest? That is where next time's preparation goes.
- ◆ What did we not need? Delete it. Checklists that grow forever stop being read.

Aviation improves because every incident is written down and shared, not because the next crew is more careful.

It is a cycle, not a project



Teams for whom upgrades are painless are not luckier. They never left phase 3.

Takeaways

1**Expect work**

An upgrade is a project with phases, not a ticket with an estimate.

3**Read the changelogs**

They are your NOTAMs, and every entry already contains its own migration.

5**Layer your tools**

Scanner, Rector, Fractor, the TCA migration check, tests, the deprecation log.
Every one has a blind spot the next one covers.

7**Stay calm**

Aviate, navigate, communicate – in that order.

2**Expect errors – and prepare for them**

You cannot prevent all of them. You can decide in advance how you will react.

4**Check deprecations in all three phases**

Today's deprecation is tomorrow's breaking change. Always.

6**Write it down and share it**

A plan in one person's head is not a plan. It is a single point of failure.

Resources

CHANGELOGS

TYPO3 Core Changelog

docs.typo3.org/c/typo3/cms-core/main/en-us/Index.html

Shipped with your core

[vendor/typo3/cms-core/Documentation/Changelog/](https://vendor.typo3.org/cms-core/Documentation/Changelog/)

Version support & ELTS

get.typo3.org – and the ELTS pages on typo3.com

TOOLS

Extension Scanner

Admin Tools › Upgrade › Scan Extension Files

netresearch/extension-scanner-cli

github.com/netresearch/t3x-nr-extension-scanner-cli

ssch/typo3-rector

github.com/sabbelasichon/typo3-rector

a9f/typo3-fractor

github.com/andreaswolf/fractor

typo3/testing-framework

github.com/TYPO3/testing-framework

This talk's hands-on workshop

[flight_ops](#) on 12.4, labs for all three phases

Check TCA Migrations / Check TCA in ext_tables.php

Admin Tools › Upgrade – no CLI equivalent

Configuration › TCA backend module

[EXT:lowlevel](#) – shows the migrated values

Thank you

Questions, war stories, and disagreements welcome.

Stefan Bürk stefan@buerk.tech · TYP03 Core Team — Framework Maintainer